DOCUMENT

17

Recording Runbook

Step by step: from running the app to a finished video, with what to say

A step-by-step guide for recording the 8 to 10 minute demo yourself. It tells you what to prepare, how to record, what to show on screen at each moment, exactly what to type, and what to say. Follow it top to bottom once as a rehearsal, then again with recording on.

Target length: **9 minutes 30 seconds**. The assignment asks for 8 to 10, and something must run live. In this plan, five of nine segments run something live.

Part A. Prepare (about 20 minutes, do this the day before)

A1. Open the project

1. Open PowerShell.
2. Go to the repository folder (the one that contains `backend`, `e2e`, `docs`, `AGENTS.md`):

```
cd <path to the repository>
```

3. Check you are in the right place: `dir` should list `AGENTS.md`, `README.md`, `backend`, `e2e`, `specs`, `docs`.

A2. Install and verify everything once

Run these one by one. Each must finish without errors.

```
pip install -e "backend[dev]"
python -m pytest backend/tests -q
```

Expected: `177 passed`.

```
cd e2e
npm install
npx playwright install chromium
npm test
cd ..
```

Expected: `1 passed`.

```
wsl -l -v
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh rehearsal
Remove-Item docs\mutation\rehearsal.txt
```

Expected: `Ubuntu-22.04` is listed; the run takes about 20 seconds and ends with `Mutation score: 100.0%`. The last command deletes the rehearsal report so it does not end up in your commits.

A3. Check you can run the app

```
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
```

Open http://localhost:5000 in a browser. You should see **Sign in**. Stop the server with `Ctrl+C`. (Restarting the server resets all data to the demo data, which is exactly what you want before recording.)

Demo accounts (password `ChangeMe123!` for all): `demo.manager`, `demo.director`, `demo.director2`, `demo.admin`.

A4. Prepare a notes file with the long commands

Typing long commands on camera invites mistakes. Create a text file called `commands.txt` outside the repository and keep it open on a second screen or in a window you do not share. Paste these into it:

```
1. python -m pytest backend/tests -q
2. python -m pytest backend/tests/test_approval_policy.py -v -k "same_user or second_approver"
3. python -m pytest backend/tests -q -x
4. wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh demo
5. cd e2e; npx playwright test --headed; cd ..
6. python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
7. Get-Content logs\ai-interactions.jsonl | Select-String '"event": "CheckSummary"' | Select-
Object -Last 7 | ForEach-Object { $r = $_.Line | ConvertFrom-Json; "{0}  {1}" -f
$r.ts.Substring(0,19), $r.summary }
8. (Get-Content logs\ai-interactions.jsonl).Count
```

A5. Open these files in your editor (VS Code) in this order, as tabs

1. `README.md` (the "Approval limits by role" table)
2. `AGENTS.md`
3. `ASSIGNMENT.md` (section 7, "Does the context file matter?")
4. `specs/001-invoice-approval-limits/spec.md`
5. `specs/001-invoice-approval-limits/tasks.md`
6. `backend/tests/test_approval_service.py` (search for `assert_refused_and_unchanged`)
7. `backend/src/invoice_app/domain/approval_policy.py` (the line `needs_director = self._limits.manager < attempt.amount <= ...` in `ManagerLimitRule`)
8. `docs/mutation/baseline.txt`
9. `docs/mutation/after.txt`
10. `docs/pr-review.md`

Also have the `.claude` folder expanded in the editor's file explorer.

A6. Make the break demo safe

In segment 4 you deliberately break one character in `approval_policy.py` and then undo it. Protect yourself first:

- **If your work is committed:** `git status` should be clean. You will undo with `git restore backend/src/invoice_app/domain/approval_policy.py`.
- **If it is not committed yet:** make a backup and keep the restore command ready:

```
Copy-Item backend\src\invoice_app\domain\approval_policy.py $env:TEMP\approval_policy.bak
```

Restore with:

```
Copy-Item $env:TEMP\approval_policy.bak backend\src\invoice_app\domain\approval_policy.py -Force
```

A7. Dress rehearsal

Do the whole of Part D once without recording, with a timer. Aim for 9 to 10 minutes. If you are over, shorten segments 2 and 3 (they are the least live).

Part B. Set up your screen (5 minutes, just before recording)

1. **Close everything else:** email, chat, notifications, other browser tabs. Turn on **Focus assist / Do not disturb** so no pop-ups appear.
2. **Resolution:** 1920 x 1080 is ideal. If your screen is larger, the text will look tiny in the video.
3. **Make text big:** - PowerShell or Windows Terminal: font size 18 or more (`Ctrl` + mouse wheel to zoom). - VS Code: `Ctrl` + `+` until the code is easily readable (about 150% zoom). - Browser: zoom to 110% to 125% (`Ctrl` + `+`).
4. **Short prompt (hides your folder path):**

```
function prompt { "PS> " }
```

5. **Clear the terminal:** `cls`.
6. **Nothing on port 5000.** Check:

```
Get-NetTCPConnection -LocalPort 5000 -ErrorAction SilentlyContinue
```

Expected: no output. If something is listed, stop that process (or close the other terminal running the server). 7. **Microphone:** use a headset if you can. Speak at a normal volume; avoid fan noise or an echoing room. 8. **Windows layout:** have the editor and the PowerShell window both ready and switch with `Alt+Tab`. Keep the browser as a third window.

Part C. Record with Microsoft Teams (3 minutes)

Teams can record a meeting that only you are in. This produces an MP4.

1. Open Microsoft Teams.

2. Go to **Calendar** and choose **Meet now** (or **Meet** then **Start meeting**). Give it a name such as "AI in SDLC demo".
3. Choose **Start meeting** and then **Join now**. Check your microphone is on (the mic icon is not crossed out). Camera can be off.
4. Click **Share** (the screen-share icon) and choose **Screen** (the whole screen), so you can switch between editor, terminal and browser. If you only use one window, choose that window instead.
5. Click **More** (the three dots) then **Record and transcribe** then **Start recording**. Wait for the "Recording started" banner.
6. Wait 3 seconds, then begin segment 1.
7. When you finish: **More** then **Record and transcribe** then **Stop recording**, then leave the meeting.
8. The recording appears in the meeting chat after a few minutes, and is saved in your OneDrive `Recordings` folder. Open it, check it plays with sound, and use **Share** to copy a link.

Menu names change slightly between Teams versions. If you cannot find "Record and transcribe", look under **More** or **Meeting options**.

Alternatives that also produce MP4:

- **Xbox Game Bar:** press `Win` + `Alt` + `R` to start and stop. It records one application window (not the whole desktop), so use it only if you keep everything in one window. Files go to `Videos\Captures`.
- **OBS Studio:** free, records the whole screen; set the output format to MP4.

Part D. The recording, segment by segment

Legend: **Show** = what is on screen. **Do** = what you type or click. **Say** = what to say (in your own words; the text is a guide, not a teleprompter). Numbers are written the way you say them.

Speak at about 150 words a minute, pause after each command so the output is visible, and never apologise for a pause: silence while a test runs is fine.

Segment 1. Introduction (0:00 to 0:45)

Show: the README, scrolled to the "Approval limits by role" table.

Do: nothing yet. Just let the table be visible.

Say: "Hello, this is my AI in SDLC assignment. The feature is approval limits by role for an invoice approval service. A Manager can approve invoices up to fifty thousand. A Director can approve up to five hundred thousand. Anything above that needs two different approvers, and the second one must be a Director. A refused attempt must give a clear reason and change nothing. The original project is C sharp and I have no dot net on this machine, so I ported the backend to Python with FastAPI, kept the same layers, and kept the original tests green before I touched the feature. In this video I will run the tests, the mutation tests and the end to end test live."

Segment 2. Context, skills, agents, hooks (0:45 to 1:45)

Show: `AGENTS.md`, then the `.claude` folder in the file explorer.

Do: 1. In `AGENTS.md` scroll slowly to the "Approval rules" table (2 seconds), then to "Review expectations" (2 seconds). 2. Click into `.claude`, expand `agents`, `skills`, `hooks` one at a time.

Say: "First, context. AGENTS dot M D is what an AI agent reads when it arrives with no knowledge of the project. It holds the approval rules and the boundaries to test, the rule that business logic lives only in the domain layer, the test commands, and my review expectations. For example, a refusal test must check the reason and that the invoice is unchanged. A test that only checks the exception type does not count. In the dot claude folder there are three skills, five agents and the hooks. The agents are a spec writer, an implementer that does one step at a time, a test reviewer, a pull request reviewer and a mutation analyst. The hooks write every prompt, tool call, sub agent result and test run into one log file, in one format."

Optional (10 seconds): open `ASSIGNMENT.md` section 7 and say: "I also tested whether this file matters. An agent with no context still planned well, but with it the plan had cent level boundaries, username normalisation, the Admin rule and the exact assertions. The comparison, including a first attempt I had to throw away, is in the write up."

Segment 3. Specification (1:45 to 2:30)

Show: `specs/001-invoice-approval-limits/spec.md`, then `tasks.md`.

Do: scroll to "Clarifications" in the spec and pause for 3 seconds; open `tasks.md` and scroll to "Phase 2" and "Phase 3".

Say: "The specification comes from Spec Kit. The clarifications section is where I settled the ambiguous points: the limits are inclusive, so exactly fifty thousand is allowed for a Manager; the first approver above five hundred thousand may be a Manager or a Director; and the same user means the same username, ignoring case and spaces. The tasks are split into two stories, single approver first and then the two approver rule, and I built them in that order."

Segment 4. Unit tests, live (2:30 to 4:00)

Show: PowerShell, large font.

Do, step by step:

1. Paste command 1 and press Enter:

```
python -m pytest backend/tests -q
```

2. When it finishes (about one second), paste command 2:

```
python -m pytest backend/tests/test_approval_policy.py -v -k "same_user or second_approver"
```

3. Switch to the editor, open `backend/tests/test_approval_service.py`, press `Ctrl+F`, search `assert_refused_and_unchanged`, and let the function be visible.
4. Prove the tests can fail. Switch to `approval_policy.py`, find `needs_director = self._limits.manager < attempt.amount <= self._limits.director` (in `ManagerLimitRule`), change the first `<` to `<=`, press `Ctrl+S`.
5. Switch to PowerShell and run command 3:

```
python -m pytest backend/tests -q -x
```

6. Show the red failure for a few seconds, then restore: - committed: `git restore backend/src/invoice_app/domain/approval_policy.py` - not committed: the `Copy-Item` restore command from step A6.
7. Run command 1 again and show the green result.

Say (while step 1 runs and after): "One hundred seventy seven tests, all green in a few seconds. They cover the pure policy rules, the service with a fake repository, a shared contract that every repository must pass, and the HTTP API."

Say (step 2): "These are the two negative rules the assignment warns about. The same user can never be both approvers, and a Manager can never be the second approver."

Say (step 3): "Every refusal test uses this helper. It makes a copy of the invoice, expects the refusal, checks the exact reason text, checks the invoice equals the copy, and checks the repository never saved anything. A test that only checks the exception type would still pass if the code half applied the approval."

Say (steps 4 to 7): "Now I prove the tests can fail. I change one character in the Manager limit rule: less than becomes less than or equal, so exactly fifty thousand would now be refused. I run the tests and the boundary test fails straight away. I restore the file, and everything is green again. That is why every limit is tested one cent either side."

Segment 5. Mutation testing, live (4:00 to 5:30)

Show: `docs/mutation/baseline.txt`, then PowerShell.

Do: 1. Show `baseline.txt` for 5 seconds (the line `Mutation score: 63.9%`). 2. Paste command 4 and press Enter:

```
wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh demo
```

3. Wait about 20 seconds while you talk. When the block appears, point at `Killed : 132` and `Mutation score: 100.0%`.
4. Clean up (you can do this after you stop recording): `Remove-Item docs\mutation\demo.txt`.

Say (before running): "Passing tests are not proof that the tests are good. Mutation testing changes the code in small ways, for example flipping a comparison, and checks that some test notices. Before the feature, with only the seven original tests, the score was sixty three point nine percent. I run it live now on the finished code. It runs under WSL because the tool needs the fork system call."

Say (while it runs, about 20 seconds): "There are now one hundred thirty two mutants, because the new policy module and its rule classes are in scope, so the numbers are not like for like, but the scope is the whole domain package in both runs."

Say (result): "One hundred thirty two of one hundred thirty two killed, one hundred percent. It did not start there. My first pass after the feature was eighty one percent with sixteen survivors, and all sixteen were missing tests: a fake repository that never actually saved, an unasserted approval timestamp, and

unasserted error messages. I added a test for each. The caveat is that this measures the domain unit tests only, not the API layer."

Segment 6. Playwright end-to-end test, live (5:30 to 6:30)

Show: PowerShell, then the browser window Playwright opens.

Do: 1. Make sure no server is running on port 5000. 2. Paste command 5 and press Enter:

```
cd e2e; npx playwright test --headed; cd ..
```

3. Watch the browser: sign-in, Approvals tab, click on Approve, the red message.
4. When the terminal shows `1 passed`, say so.

Say (before): "Here is the one end to end test the assignment asks for: a rejected over-limit approval through the real user interface. It uses page objects and role based locators, no CSS selectors."

Say (while the browser runs): "It signs in as the demo Manager, opens Approvals, presses Approve on the seventy five thousand invoice, expects the exact refusal message on screen, expects the invoice to still be listed as pending, and then checks the invoices table still says Pending Approval. Refused, explained, and nothing changed."

Say (after): "One test passed."

Segment 7. The feature by hand in the UI, live (6:30 to 7:45)

Show: browser at http://localhost:5000.

Do: 1. Start the app: paste command 6 into PowerShell and wait for `Application startup complete`. 2. Open the browser (zoom 110 to 125 percent) and go to http://localhost:5000. 3. Follow this table slowly:

Step	Action	Result to point at	Say
1	Sign in as `demo.manager` / `ChangeMe123!`, open the **Approvals** tab, press **Approve** on INV-2001	Red message about the Manager limit; INV-2001 is still in the list	"As the Manager, over my limit: refused with a reason, and the invoice is unchanged."
2	**Sign out**, sign in as `demo.director`, **Approvals**, press **Approve** on INV-2001	INV-2001 disappears from the list	"A Director can approve it alone."
3	Still `demo.director`, press **Approve** on INV-2002	The entry now says "1 of 2 approvals" and stays in the list	"Above five hundred thousand, one approval is not enough, so it stays pending and shows one of two."
4	Press **Approve** on INV-2002 again	Red message: already approved	"The same user cannot be both approvers, so it is refused."

Step	Action	Result to point at	Say
5	**Sign out**, sign in as `demo.director2`, **Approvals**, **Approve** INV-2002, then open the **Invoices** tab	INV-2002 shows Approved	"A different Director completes it, and the invoice is approved."

4. Stop the server with `Ctrl+C` in PowerShell.

Optional if you are under time: restart the server, sign in as `demo.manager`, and show a Manager refused as the second approver on a 600,000 invoice that already has one approval. (A Director must give the first approval, then sign out and use the Manager.)

Segment 8. Review and verdicts (7:45 to 8:45)

Show: the pull request description on GitHub if the PR is open; otherwise `docs/pr-review.md`.

Do: scroll the findings table slowly. Pause on F1 and F2.

Say: "I reviewed my own diff with the pull request reviewer agent and judged every finding. There were eleven: six fixed, three dismissed with reasons, and two deferred with a reason and a tracking task. Two were real bugs that I would not have found by reading my own code. First, two simultaneous approvals by the same Director both passed the already approved check, which breaks the central rule. I fixed it with a lock, and wrote race tests that fail when the lock is removed. My first version of those tests passed even without the lock, so they proved nothing, and I rewrote them. Second, an absurdly large amount was accepted and then broke the invoice list for everyone. It is now validated and rejected with a message. Dismissed findings have a written reason, for example role names are matched exactly so a mistyped role fails closed. Deferred ones point to tasks twenty four and twenty five."

Segment 9. AI log and wrap-up (8:45 to 9:30)

Show: PowerShell.

Do: paste command 7, then command 8:

```
Get-Content logs\ai-interactions.jsonl | Select-String '"event": "CheckSummary"' | Select-Object -Last 7 | ForEach-Object { $r = $_.Line | ConvertFrom-Json; "{0}  {1}" -f $r.ts.Substring(0,19), $r.summary }
(Get-Content logs\ai-interactions.jsonl).Count
```

Then show `ASSIGNMENT.md` for the closing line.

Say: "All AI interaction is logged by hooks into one file. Each line is a JSON object with the same 27 keys every time: the tool and model, the agent and its mode, the prompt, skills, sub-agents and tools used, what the AI suggested, the human decision, how it was verified, the status and the files touched. Test runs become check summary lines with pass or fail, so you can see the deliberate failure from earlier and the green runs around it. The first few lines are marked reconstructed because they cover planning before the hooks existed. To wrap up: unit tests for every rule including both negative rules; a mutation score from sixty four to one hundred percent; one Playwright test; a spec made with Spec Kit; hooks and a log; and a review where every finding has a verdict. One honest limit: the project agents were not selectable by name

in the session that created them, so three roles ran as general purpose sub agents following the agent files. Thank you for watching."

Stop the recording.

Part E. After recording (10 minutes)

1. Watch the whole video once at normal speed. Check: sound is clear, terminal text is readable, the run is between 8 and 10 minutes, nothing private is visible (email, chat, file paths with your name).
2. Clean up:

```
Remove-Item docs\mutation\demo.txt -ErrorAction SilentlyContinue
git status
```

`git status` must not show a changed `approval_policy.py`. If it does, restore it (step A6) and run the tests again. 3. Upload the video (or copy the Teams or OneDrive share link). Open the link in a private browser window to confirm a reviewer can open it. 4. Paste the link into `ASSIGNMENT.md`, replacing the line `_TODO: paste the link here after recording (8-10 min)._` 5. Rebuild the PDFs: `python scripts/build_docs.py`.

Part F. If something goes wrong while recording

Problem	What to do
A command fails or you mistype	Say "let me run that again" and run it. Do not stop the recording
Mutation run is slower than a minute	Keep talking about the survivors; if it is still running after 90 seconds, show `docs/mutation/after.txt` and say that you are showing the saved result
The manual walkthrough finds INV-2001 already approved	An earlier run approved it. Restart the server to reset the data. (The Playwright test makes its own invoice and does not depend on it.)
Port 5000 is busy	`Get-NetTCPConnection -LocalPort 5000`, then stop that process
You forgot to restore `approval_policy.py`	Restore it on camera and run the tests: it is one more proof
You go over 10 minutes	Skip the optional parts and cut segment 3 to 15 seconds
You are under 8 minutes	Add the optional context comparison in segment 2 and the Manager-as-second-approver step in segment 7
Teams shows "Recording not available"	Use Xbox Game Bar (`Win` + `Alt` + `R`) or OBS instead

Part G. One-page cheat sheet (print this)

Time	Do	Say (key point)
0:00	README table	Feature, Python port, what will run live
0:45	`AGENTS.md`, `.claude`	Context, 3 skills, 5 agents, hooks, one log
1:45	spec and tasks	Clarifications, two stories in order
2:30	`pytest` x2, helper, break `<` to `<=`, restore	177 green; negative rules; unchanged-invoice helper; test fails on one character
4:00	baseline, then mutation run	63.9% before; live run; 132 of 132; why survivors existed
5:30	`npx playwright test --headed`	Rejected over-limit approval through the UI
6:30	start app, 5 UI steps	Manager refused; Director alone; 1 of 2; same user refused; second Director
7:45	PR or `pr-review.md`	11 findings, 6 fixed, 3 dismissed, 2 deferred; the race and the huge amount
8:45	log query, wrap-up	One format log; summary; honest limit